

# Policy-based Reinforcement Learning: REINFORCE, Actor-Critic, A2C

Biên soạn: ThS. Phan Trung Phát  
Liên hệ: [phatpt@uit.edu.vn](mailto:phatpt@uit.edu.vn)

Tái bản lần 1 -- Tháng 03/2026

Lưu hành nội bộ

## A. MỤC TIÊU

- Nắm rõ và sử dụng được một số thành phần cơ bản của PyTorch.
- Tích hợp toàn bộ thuật toán DQN với các môi trường có sẵn trên Gymnasium và môi trường tùy chỉnh.
- Tìm hiểu về quá trình huấn luyện và đào tạo Agent, hiểu được các hyperparameters và cách tùy chỉnh để đạt được kết quả hiệu quả trên giải thuật DRL.
- Đánh giá và so sánh được chính sách được xây dựng dựa trên DQN và các policy khác.
- Tìm hiểu Stable-Baselines3 và tích hợp DQN một cách đơn giản.

## B. GIỚI THIỆU

### 1. Từ value-based đến policy-based Reinforcement Learning

#### a. Giới hạn của DQN và các value-based nói chung

Trong học tăng cường, quá trình chuyển từ value-based sang policy-based RL xuất phát từ một vấn đề rất cơ bản: Agent không chỉ cần biết “hành động nào có giá trị cao”, mà cuối cùng nó phải học được cách hành động hiệu quả trong môi trường. value-based methods, chẳng hạn như Q-learning hoặc DQN, tiếp cận bài toán bằng cách học một hàm giá trị hành động  $q(s, a)$ . Hàm này cho biết nếu Agent đang ở trạng thái  $s$  và chọn hành động  $a$ , thì về dài hạn hành động đó có thể đem lại tổng phần thưởng kỳ vọng cao đến mức nào.

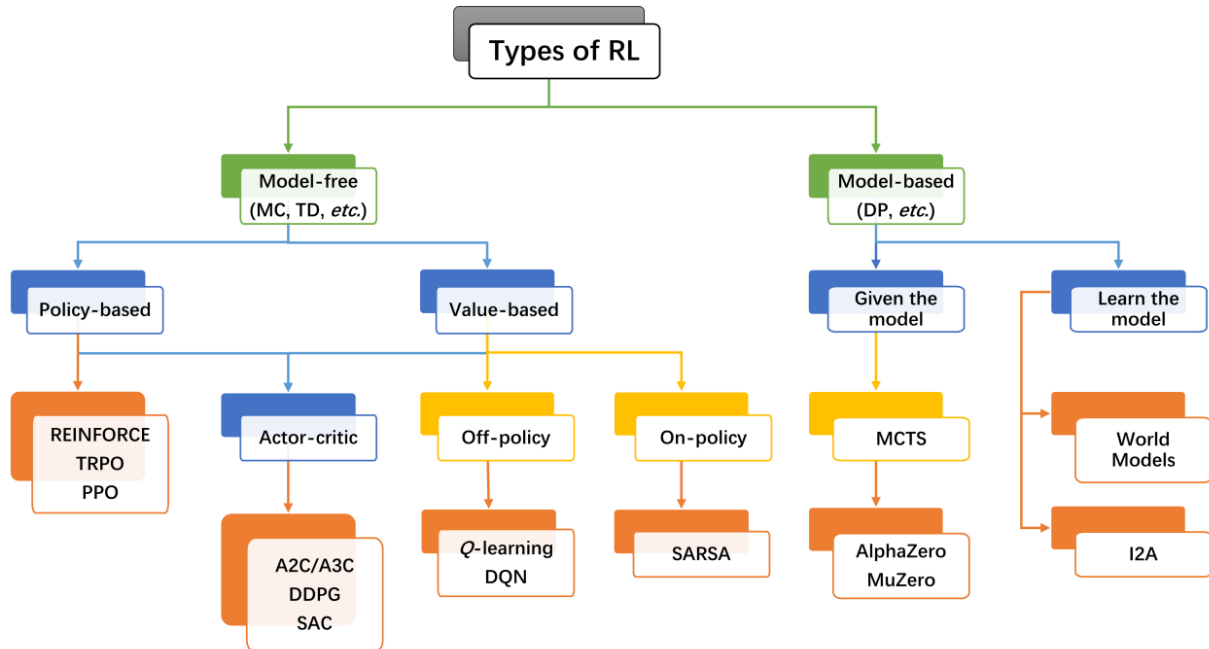
#### \*Vấn đề với continuous action space:

Sau khi học được hàm  $q(s, a)$ , Agent sẽ chọn hành động bằng cách lấy hành động có giá trị lớn nhất:

$$a^* = \underset{a}{\operatorname{argmax}} \hat{q}(s, a, \theta)$$

Cách làm này rất trực quan. Nếu Agent biết hành động nào có giá trị cao nhất, nó chỉ cần chọn hành động đó. Với các bài toán có số lượng hành động nhỏ và rời rạc, cách tiếp cận này

hoạt động khá tốt. Ví dụ, trong một trò chơi đơn giản, Agent chỉ có bốn hành động: đi lên, đi xuống, đi trái và đi phải. Khi đó, việc tính giá trị của từng hành động rồi chọn hành động tốt nhất là hoàn toàn khả thi.



Hình 1. Phân loại các giải thuật RL<sup>1</sup>

Tuy nhiên, hạn chế lớn bắt đầu xuất hiện khi không gian hành động trở nên lớn hoặc liên tục (large / continuous action spaces). Trong nhiều bài toán thực tế, hành động không còn là một tập nhỏ các lựa chọn cố định. Chẳng hạn, trong bài toán điều chỉnh công suất truyền tín hiệu, Agent có thể phải chọn một mức công suất bất kỳ trong khoảng từ 0 đến 1. Điều đó có nghĩa là hành động có thể là 0.1, 0.25, 0.337, 0.891, và vô số giá trị khác. Khi action space là liên tục, phép chọn hành động bằng *argmax* trở nên rất khó, vì Agent không thể duyệt qua vô hạn hành động để tìm hành động tốt nhất. Kể cả khi đã sử dụng DQN để ước lượng thì cũng rất khó để duyệt toàn bộ không gian, lúc này bài toán đã trở thành một vấn đề tối ưu

<sup>1</sup> Liu X, Zhang J, Hou Z, Yang YI, Gao YQ. From predicting to decision making: Reinforcement learning in biomedicine. WIREs Comput Mol Sci. 2024;14(4):e1723. <https://doi.org/10.1002/wcms.1723>

phi tuyến trong không gian liên tục rất khó khăn. Từ đây, có thể gây ra nhiều vấn đề như có thể có nhiều điểm tối đa hoá cục bộ hay tốn chi phí tính toán cực lớn...

Đây là điểm bất lợi cốt lõi của value-based RL. Nó phụ thuộc mạnh vào khả năng so sánh giá trị của các hành động. Khi số lượng hành động nhỏ, việc so sánh này đơn giản. Nhưng khi số lượng hành động rất lớn hoặc liên tục, việc tìm hành động tối ưu trở nên tốn kém, không ổn định hoặc thậm chí không khả thi. Vì vậy, value-based methods như DQN thường phù hợp hơn với các bài toán có discrete action space, nhưng gặp khó khăn trong các bài toán continuous control.

### \*DQN không học cách hành động, mà học cách đánh giá hành động:

Một hạn chế khác của value-based methods là chúng không học policy một cách trực tiếp. Mục tiêu cuối cùng của học tăng cường là tìm ra một chính sách hành động tốt, tức là cách quyết định Agent nên làm gì trong từng trạng thái. Tuy nhiên, value-based methods lại đi theo hướng gián tiếp: trước hết học hàm giá trị  $q(s, a)$ , sau đó suy ra policy từ hàm giá trị đó. Nói cách khác, policy không phải là đối tượng được tối ưu trực tiếp, mà chỉ là kết quả được rút ra sau khi đã ước lượng value function.

Điều này tạo ra một rủi ro quan trọng. Nếu hàm  $q(s, a)$  được ước lượng không chính xác, policy được suy ra từ nó cũng có thể sai. Ví dụ, nếu Agent ước lượng nhầm rằng hành động  $a_1$  tốt hơn hành động  $a_2$ , nó sẽ chọn  $a_1$ , mặc dù trong thực tế  $a_2$  có thể đem lại phần thưởng dài hạn cao hơn. Như vậy, sai số trong value estimation có thể trực tiếp dẫn đến sai lầm trong decision making.

### \*Hạn chế trong exploration:

Ngoài ra, value-based methods thường gặp hạn chế trong việc exploration. Trong DQN, exploration thường được thực hiện bằng epsilon-greedy. Nghĩa là Agent phần lớn thời gian

sẽ chọn hành động có giá trị lớn nhất, nhưng thỉnh thoảng sẽ chọn ngẫu nhiên một hành động khác để khám phá. Cách này đơn giản và dễ triển khai, nhưng tương đối thô. Nó không biểu diễn được một cách mềm dẻo mức độ tin tưởng của Agent vào từng hành động.

Ví dụ, giả sử tại một trạng thái  $s$ , agent ước lượng ba hành động như sau:

$$q(s, a_1) = 10, \quad q(s, a_2) = 9.9, \quad q(s, a_3) = 9.8$$

Theo cơ chế chọn hành động greedy, Agent gần như luôn chọn  $a_1$ , mặc dù  $a_2$  và  $a_3$  cũng có giá trị rất gần. Nếu dùng epsilon-greedy, Agent có thể chọn ngẫu nhiên  $a_2$  hoặc  $a_3$ , nhưng việc chọn này không dựa trên một phân phối xác suất được học một cách có ý nghĩa. Điều này làm cho exploration trong value-based methods kém linh hoạt hơn so với policy-based methods.

### b. Policy-based RL: sự chuyển dịch tư duy

Từ các hạn chế trên, policy-based RL được đưa ra như một hướng tiếp cận trực tiếp hơn. Thay vì học hàm giá trị hành động  $q(s, a)$ , Agent học trực tiếp một policy:

$$\pi(a | s, \theta)$$

Trong đó,  $\pi(a | s, \theta)$  biểu diễn xác suất chọn hành động  $a$  khi Agent đang ở trạng thái  $s$ , còn  $\theta$  là các tham số của policy, thường là trọng số của một mạng nơ-ron. Với cách tiếp cận này, Agent không cần phải tính giá trị của mọi hành động rồi chọn hành động tốt nhất. Thay vào đó, policy trực tiếp sinh ra hành động hoặc phân phối xác suất trên các hành động.

Điểm khác biệt quan trọng nằm ở tư duy tối ưu. value-based methods trả lời câu hỏi: “Hành động này có giá trị bao nhiêu?”. Sau đó, Agent chọn hành động có giá trị cao nhất. Trong khi đó, policy-based methods trả lời trực tiếp hơn: “Trong trạng thái này, Agent nên hành động như thế nào?”. Điều này giúp policy-based RL phù hợp hơn với các bài toán mà việc đánh giá toàn bộ action space là khó khăn.

## 2. Những yếu tố của policy-based RL

### \*Discrete action space:

Đối với discrete action space, policy có thể xuất ra một phân phối xác suất. Ví dụ, nếu Agent có ba hành động, policy có thể cho ra:

$$\pi(a | s) = [0.1, 0.7, 0.2]$$

Điều này có nghĩa là Agent chọn hành động thứ hai với xác suất cao nhất, nhưng vẫn có khả năng chọn các hành động khác. Nhờ đó, exploration trở thành một phần tự nhiên của policy, thay vì phải thêm vào bằng một cơ chế bên ngoài như epsilon-greedy.

### \*Continuous action space:

Đối với continuous action space, policy-based methods còn thể hiện ưu thế rõ hơn. Policy có thể trực tiếp sinh ra một giá trị hành động liên tục. Ví dụ, trong bài toán điều khiển công suất, policy có thể được biểu diễn dưới dạng:

$$a = \mu(s, \theta)$$

Nếu trạng thái  $s$  mô tả tình trạng kênh truyền, mức nhiễu và yêu cầu băng thông, mạng nơ-ron có thể trực tiếp xuất ra mức công suất phù hợp, chẳng hạn  $P = 0.63$ . Agent không cần duyệt qua hàng nghìn hay vô hạn mức công suất khác nhau để tìm giá trị tốt nhất. Đây là lý do policy-based methods được xem là tự nhiên hơn cho continuous control.

### \*Learning Objective:

Mục tiêu học trong policy-based RL cũng được định nghĩa trực tiếp trên hiệu quả của policy. Ta thường định nghĩa hàm mục tiêu:

$$J(\theta) = \mathbb{E}[\text{total reward}]$$

Mục tiêu là tìm bộ tham số  $\theta$  sao cho policy tạo ra tổng phần thưởng kỳ vọng lớn nhất:

$$\max_{\theta} J(\theta)$$

Điều này cho thấy policy-based methods tối ưu trực tiếp hiệu quả hành vi của Agent. Thay vì cố gắng học chính xác toàn bộ value function, Agent điều chỉnh policy theo hướng làm tăng tổng reward.

### \*Policy Gradient:

Một công thức quan trọng trong Policy Gradient là:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}[G_t \nabla_{\theta} \ln \pi(a_t | s_t, \theta)]$$

Công thức này có thể hiểu một cách trực quan như sau: nếu một hành động được chọn trong trạng thái nào đó dẫn đến kết quả tốt, tức là return  $G_t$  cao, ta tăng xác suất chọn hành động đó trong tương lai. Ngược lại, nếu hành động dẫn đến kết quả xấu, xác suất chọn hành động đó sẽ giảm. Như vậy, policy được cập nhật trực tiếp dựa trên trải nghiệm của agent.

### \*Update rule:

Và cuối cùng là cập nhật lại rule:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \ln \pi(a_t | s_t, \theta)$$

Ví dụ đơn giản, giả sử một agent đang học cách chọn đường đi trong mạng. Tại một node, nó có ba lựa chọn next-hop:  $a_1$ ,  $a_2$ , và  $a_3$ . Ban đầu, policy có thể chọn các actions với xác suất gần như nhau. Sau một số lần thử, Agent nhận thấy rằng chọn  $a_2$  thường dẫn đến đường truyền có độ trễ thấp hơn và reward cao hơn. Khi đó, policy gradient sẽ điều chỉnh tham số  $\theta$  để tăng  $\pi(a_2 | s)$ , tức là tăng xác suất chọn  $a_2$  trong trạng thái tương tự sau này.

Như vậy, quá trình chuyển từ value-based sang policy-Based RL là một sự chuyển dịch từ việc “đánh giá hành động” sang việc “học cách hành động”. Value-based methods tập trung vào việc ước lượng giá trị của từng hành động, sau đó dùng giá trị này để chọn hành

động. Policy-based methods thì học trực tiếp cơ chế chọn hành động, nhờ đó linh hoạt hơn trong các môi trường phức tạp.

Tuy nhiên, điều này không có nghĩa policy-based methods hoàn toàn thay thế value-based methods trong mọi trường hợp. Policy-based methods cũng có nhược điểm, chẳng hạn như gradient có thể có phương sai cao, quá trình học có thể không ổn định và cần nhiều mẫu tương tác với môi trường để hội tụ. Trong khi đó, value-based methods thường có độ ổn định tốt hơn trong quá trình học và có sample efficiency tốt hơn. Chính vì vậy, các phương pháp hiện đại thường kết hợp cả hai hướng tiếp cận, tiêu biểu là Actor-Critic.

### 3. REINFORCE: thuật toán cơ bản trong nhóm policy-based RL

REINFORCE có thể được xem như một thuật toán cơ bản và quan trọng nhất trong nhóm policy-based RL, cụ thể là thuộc loại **Monte Carlo Policy Gradient methods**. Đây chính là thuật toán cụ thể đầu tiên giúp biến những ý tưởng lý thuyết của **Policy Gradient** thành một quy trình học thực tế.

---

#### Algorithm 1 REINFORCE (Monte Carlo Policy Gradient)

---

```

1: Initialise policy parameters  $\theta$ 
2: for each episode do
3:   Generate trajectory  $\tau = (s_1, a_1, r_1, \dots, s_T, a_T, r_T)$  using  $\pi(a|s, \theta)$ 
4:   Compute returns  $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$  for  $t = 1, \dots, T$ 
5:   for  $t = 1$  to  $T$  do
6:      $\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \ln \pi(a_t | s_t, \theta)$ 
7:   end for
8: end for

```

---

Hình 2. Pseudocode thuật toán REINFORCE

Về nguyên lý, REINFORCE hoạt động dựa trên một ý tưởng rất trực quan:

| Nếu một hành động trong trajectory dẫn đến kết quả tốt, ta tăng xác suất của hành động đó. Nếu kết quả kém, ta giảm xác suất của nó.



Điều này hoàn toàn phù hợp với trực giác mà chúng ta đã thảo luận trong phần **Policy Gradient** trước đó.

Cụ thể, REINFORCE hoạt động theo một quy trình rất rõ ràng:

- Trước hết, Agent sử dụng policy hiện tại để tương tác với môi trường và sinh ra một trajectory, bao gồm chuỗi trạng thái, hành động và phần thưởng  $\tau$ .
- Sau khi episode kết thúc, agent sẽ tính toán **return**  $G_t$  cho từng thời điểm, tức là tổng phần thưởng từ thời điểm đó trở đi. Return này đóng vai trò như một thước đo để đánh giá “hành động tại thời điểm  $t$  tốt đến mức nào trong dài hạn”.
- Tiếp theo, Agent sử dụng giá trị  $G_t$  để cập nhật policy.

Về trực giác của update rule được hiểu một cách đơn giản như sau:

- ◇ Nếu  $G_t > 0$ : hành động  $a_t$  là tốt  $\rightarrow$  tăng xác suất  $\pi(a_t | s_t)$
- ◇ Nếu  $G_t < 0$ : hành động  $a_t$  là kém  $\rightarrow$  giảm xác suất của nó

Như vậy, policy dần dần “học” từ trải nghiệm bằng cách củng cố những hành động tốt và loại bỏ những hành động xấu. Tuy nhiên, mặc dù rất trực quan nhưng REINFORCE cũng có những hạn chế quan trọng. Do sử dụng return  $G_t$ , vốn có thể biến động rất mạnh giữa các episode, quá trình cập nhật trở nên không ổn định và có phương sai cao. Ngoài ra, việc phải chờ đến khi kết thúc episode mới cập nhật khiến việc học chậm và kém hiệu quả về mặt dữ liệu. Vì thế, REINFORCE thường là baseline cho các phương pháp cải tiến sau này.

#### 4. Giới thiệu về Actor-Critic

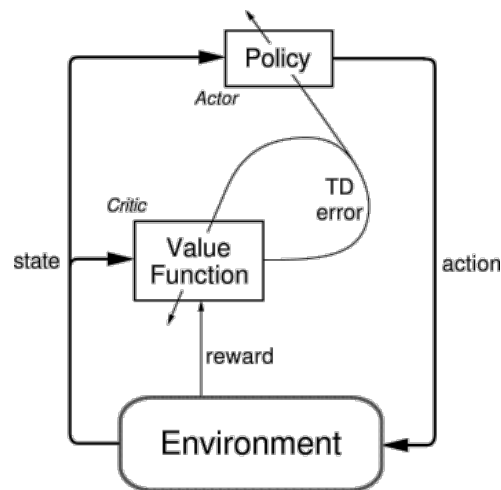
Sau khi đã nhận thấy rõ về những hạn chế của REINFORCE, một hướng giải quyết tự nhiên được đưa ra có thể kể đến là **Actor-Critic**. Đây là một phương pháp kết hợp giữa policy-based và value-based RL, với mục tiêu tận dụng ưu điểm của cả hai để tạo ra một thuật toán vừa linh hoạt, vừa ổn định hơn.

Actor-Critic được xây dựng dựa trên một ý tưởng rất trực quan: thay vì chỉ học policy như REINFORCE, ta bổ sung thêm một thành phần giúp **đánh giá hành động** để hỗ trợ quá trình học. Cụ thể, mô hình được chia thành hai phần:

- **Actor**: học policy  $\pi(a | s, \theta)$ , chịu trách nhiệm chọn hành động, giống như trong REINFORCE
- **Critic**: học value function  $V(s)$  hoặc  $Q(s, a)$ , chịu trách nhiệm đánh giá hành động

Ta có thể hiểu đơn giản:

- Actor = “người ra quyết định”
- Critic = “người chấm điểm”



Hình 3. Thuật toán Actor-Critic<sup>2</sup>

Khác với REINFORCE phải chờ đến khi kết thúc toàn bộ episode mới cập nhật, Actor-Critic có thể cập nhật ngay sau mỗi bước tương tác với môi trường. Khi Agent thực hiện một hành động và nhận được reward, Critic sẽ so sánh giữa giá trị dự đoán trước đó và giá trị thực tế sau khi quan sát kết quả. Sự khác biệt này được gọi là **Temporal Difference (TD error)**.

<sup>2</sup> <http://www.incompleteideas.net/book/ebook/node66.html>

Nếu TD error dương, nghĩa là kết quả tốt hơn mong đợi, hành động vừa thực hiện là tốt; nếu TD error âm, nghĩa là kết quả kém hơn mong đợi, hành động đó không hiệu quả.

Dựa trên tín hiệu này, Actor sẽ điều chỉnh policy theo hướng rất giống với REINFORCE, nhưng thay vì dùng toàn bộ return  $G_t$ , nó dùng TD error. Điều này mang lại một cải tiến quan trọng: tín hiệu cập nhật trở nên **ổn định hơn và ít nhiễu hơn**, vì nó chỉ phụ thuộc vào bước hiện tại và trạng thái kế tiếp, thay vì toàn bộ tương lai dài hạn. Nhờ đó, Actor-Critic giảm đáng kể vấn đề phương sai cao mà REINFORCE gặp phải.

---

```

1: Initialise actor parameters  $\theta$ 
2: Initialise critic parameters  $w$ 
3: for each step do
4:   Observe state  $s_t$ 
5:   Select action  $a_t \sim \pi(a|s_t, \theta)$ 
6:   Observe reward  $r_t$  and next state  $s_{t+1}$ 
7:   Compute TD error:  $\delta_t = r_t + \gamma \hat{v}(s_{t+1}, w) - \hat{v}(s_t, w)$ 
8:   Update critic:  $w \leftarrow w + \beta \delta_t \nabla_w \hat{v}(s_t, w)$ 
9:   Update actor:  $\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \ln \pi(a_t|s_t, \theta)$ 
10: end for

```

---

*Hình 4. Pseudocode thuật toán Actor-Critic*

Một lợi ích quan trọng khác là Actor-Critic giải quyết tốt hơn bài toán **credit assignment**, tức là xác định hành động nào thực sự đóng góp vào reward. Trong REINFORCE, toàn bộ return  $G_t$  có thể bị ảnh hưởng bởi những sự kiện xảy ra rất xa trong tương lai, khiến việc “đổ lỗi” cho một hành động trở nên thiếu chính xác. Trong khi đó, Actor-Critic sử dụng TD error mang tính cục bộ hơn, giúp phản ánh chính xác hơn tác động của hành động vừa thực hiện. Ngoài ra, việc cập nhật theo từng bước cũng giúp Actor-Critic **học nhanh hơn và hiệu quả hơn về mặt dữ liệu**. Agent không cần chờ hết một episode dài mới học được điều gì đó, mà có thể cải thiện policy liên tục trong quá trình tương tác. Điều này đặc biệt quan trọng trong các môi trường phức tạp hoặc có episode dài.

Và cuối cùng là thuật toán A2C được kế thừa hoàn toàn ý tưởng của Actor-Critic và thực hiện cải tiến để tiến đến hoàn thiện. A2C thực chất chia sẻ cùng một ý tưởng nền tảng, đó là sử dụng một thành phần Actor để học policy và một thành phần Critic để đánh giá giá trị trạng thái, nhưng điểm khác biệt quan trọng nằm ở cách chúng thu thập dữ liệu và cập nhật tham số. Với Actor-Critic cơ bản, quá trình học diễn ra theo kiểu online: sau mỗi bước tương tác với môi trường, Agent lập tức tính toán TD error dựa trên reward vừa nhận và giá trị dự đoán của Critic, rồi cập nhật cả Actor và Critic ngay lập tức. Cách làm này giúp thuật toán đơn giản và phản ứng nhanh, nhưng do mỗi lần cập nhật chỉ dựa trên một mẫu duy nhất, tín hiệu học thường nhiễu và dễ dao động.

$$A_t = G_t - V(s_t)$$

Ngược lại, A2C thay đổi cách hiện thực bằng cách không cập nhật theo từng bước riêng lẻ, mà thu thập một đoạn trajectory gồm nhiều bước liên tiếp (gọi là rollout), sau đó mới thực hiện cập nhật theo dạng batch. Từ các dữ liệu đã thu thập, A2C tính toán return hoặc advantage cho từng bước, rồi sử dụng các giá trị này để cập nhật Actor và Critic một cách đồng thời. Nhờ việc trung bình hóa thông tin từ nhiều bước thay vì chỉ một bước, gradient trở nên ổn định hơn và quá trình học ít bị ảnh hưởng bởi nhiễu ngẫu nhiên. Ngoài ra, A2C thường bổ sung thêm entropy vào hàm loss để khuyến khích exploration, giúp tránh việc policy hội tụ quá sớm vào một hành vi chưa tối ưu.

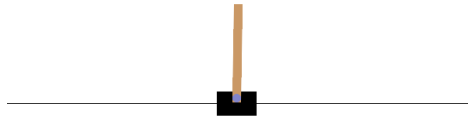
## C. THỰC HÀNH

### 1. Thực hiện hiện thực thuật toán REINFORCE trên môi trường CartPole-v1

**Mô tả:** Sinh viên tìm hiểu về các ý tưởng và cách xây dựng thuật toán trên môi trường CartPole-v1 có sẵn trên Gymnasium với 1 mạng nơ-ron đơn giản.

- **Bước 1:** Nhìn nhận bài toán.

[CartPole](#) là một môi trường kinh điển trong Gymnasium được thiết kế chủ yếu nhằm giới thiệu và kiểm chứng các thuật toán học tăng cường, trong một bài toán đơn giản nhưng vẫn mang tính bản chất của điều khiển động. Mục tiêu của môi trường là để Agent học cách đưa ra quyết định theo thời gian nhằm giữ cân bằng cho một cây gậy đặt trên xe đẩy, qua đó giúp người học hiểu rõ cách một policy được học để tối ưu hành vi dựa trên trạng thái quan sát. Nhờ cấu trúc trạng thái nhỏ, hành động đơn giản và cơ chế reward rõ ràng, CartPole thường được sử dụng như “bài toán nhập môn” để thử nghiệm các thuật toán như DQN, Policy Gradient hay Actor-Critic, đồng thời cung cấp một nền tảng trực quan để phân tích các khái niệm quan trọng như exploration, exploitation và ổn định chính sách trong học tăng cường.



- **Bước 2:** Tìm hiểu về các biểu diễn Markov Decision Process (MDP) của bài toán.

- ? *Observation Space* mô tả điều gì?
- ? *Action Space* mô tả điều gì?
- ? *Reward* được định nghĩa như thế nào?
- ? Nguyên tắc hoạt động của Agent?
- ? Khi nào thì bài toán kết thúc?

- **Bước 3:** Cài đặt thư viện liên quan: `pip install -r requirements.txt`

- **Bước 4:** Mở file **Lab4.1-st.ipynb**, thực hiện chạy *Part 1*, *Part 2* đã được cung cấp sẵn.

Cách tạo thứ 2 một mạng nơ-ron đơn giản: có 2 lớp, ở giữa là hàm kích hoạt ReLU, ở cuối là 1 lớp Softmax. Tổng quát kiến trúc: `Linear(4 → 128) → ReLU → Linear(128 → 2) → Softmax`

```
def __init__(self, input_dim=4, hidden_dim=128, output_dim=2):
```

```

super().__init__()
self.net = nn.Sequential(
    nn.Linear(input_dim, hidden_dim),
    nn.ReLU(),
    nn.Linear(hidden_dim, output_dim),
    nn.Softmax(dim=-1),
)

def forward(self, x):
    return self.net(x)

```

? Tại sao lớp cuối cùng là Softmax? Softmax giúp mô hình đạt được điều gì?

Thực hiện chọn action theo xác suất và đồng thời trả về thêm log probability cho gradient.

```

def select_action(policy_net, state):
    """Sample one action from policy distribution and return log-probability."""

    # Convert state to tensor and get action probabilities from policy network
    state_tensor = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)
    probs = policy_net(state_tensor).squeeze(0)

    # Create a categorical distribution and sample an action
    dist = Categorical(probs)
    action = dist.sample()
    return int(action.item()), dist.log_prob(action)

```

? Categorical dùng để làm gì? sample action được thực hiện như thế nào?

## 2. Hiện thực thuật toán A2C từ Stable-Baseline3 trên môi trường CartPole-v1

**Mô tả:** Sinh viên thực hiện hiện thực A2C trên môi trường CartPole-v1, thực hiện tìm hiểu policy MlpPolicy từ Stable-Baseline3 trong việc tự tích hợp A2C với các môi trường khác.

- **Bước 1:** Tiếp tục với file **Lab4.1-st.ipynb**, thực hiện chạy *Part 4* đã được cung cấp sẵn.
- **Bước 2:** Import A2C từ Stable-Baseline3 để sử dụng.

```
from stable_baselines3 import A2C
```

- **Bước 3:** Lựa chọn policy và tích hợp huấn luyện A2C trên môi trường **CartPole-v1**

```
train_env = gym.make('CartPole-v1')
train_env.reset(seed=SEED + 4000)
train_env.action_space.seed(SEED + 4000)

total_timesteps = 15000 if FAST_MODE else 50000
a2c_model = A2C('MlpPolicy', train_env, verbose=0, seed=SEED)
a2c_model.learn(total_timesteps=total_timesteps)
```

- **Bước 4:** Thực hiện chạy huấn luyện và tiến hành đánh giá, so sánh với REINFORCE, Actor-Critic.

**?** Bạn có nhận xét gì về kết quả đạt được? Đồng thời, bạn có nhận xét gì về độ ổn định qua quá trình training?

## D. YÊU CẦU & NỘI BÀI

### 1. Yêu cầu

1. Đối với *CartPole Environment*, dựa trên các đoạn mã được cung cấp, hoàn thành các block code còn lại để tích hợp thuật toán REINFORCE, Actor-Critic và A2C vào môi trường CartPole tại file **Lab4.1-st.ipynb, Phần 2,3,4**. Thực hiện vẽ reward curve của 2 thuật toán trên và tiến hành phân tích về các khía cạnh:

- Tốc độ học tập
- *reward curve* trong từng thuật toán thì đâu là thuật toán mượt hơn, điều này chứng minh được gì?

Giải thích về việc TD error giúp giảm variance trong môi trường này? Actor đang học từ đâu? Critic đang học từ đâu? Thuật toán nào mang tính ổn định nhất?

**Lưu ý:** Chạy lại với nhiều seed khác nhau và tổng hợp kết quả ( $\geq 5$ ).

2. Thực hiện tích hợp A2C từ Stable-Baseline3 trên môi trường CartPole, thực hiện so sánh các thông số average return, std return và max return, bạn có nhận định gì về 4 thuật toán: Random, REINFORCE, Actor-Critic và A2C.

3. Đọc mã nguồn của A2C từ Stable-Baseline3 để thực hiện tự hiện thực A2C đối với môi trường CartPole. Cách tính Advantage, cơ chế update batch và entropy bonus (nếu có).
4. Đối với *Edge Offloading Environment*, đọc mô tả chi tiết về môi trường tại file **note-edge-offloading.txt**, thực hiện hiện thực hoá môi trường này.
5. Thực hiện tích hợp thuật toán A2C tại câu số 3 và Actor-Critic vào môi trường Edge Offloading đã xây dựng. Thực hiện đánh giá về độ ổn định của 2 thuật toán này trong quá trình huấn luyện.
6. Xây dựng các policy: Random, Device-only, Edge-only và Greedy latency. Vẽ bảng và tiến hành so sánh, đánh giá các thông số: Avg Latency, P95 Latency, Avg Reward, Device % / Edge % và Gap to Greedy. Bạn có nhận xét gì khi so sánh giá giữa A2C và các policy này.



## 2. Yêu cầu nộp bài

- Sinh viên tìm hiểu và thực hành theo hướng dẫn. Thực hiện **theo nhóm**.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng file. Trong đó:
  - Trình bày chi tiết quá trình thực hành và trả lời các câu hỏi nếu có (kèm theo các ảnh chụp màn hình tương ứng).
  - Giải thích các kết quả đạt được.
  - Tải mẫu báo cáo thực hành và trình bày theo mẫu được cung cấp.

Nén tất cả các file và đặt tên file theo định dạng theo mẫu:

**NhomY-LabX\_MSSV1\_MSSV2**

Ví dụ: Nhom1-Lab04\_29520001\_29520002\_29520003

- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.
- Các bài nộp không tuân theo yêu cầu sẽ **KHÔNG** được chấm điểm.

**HẾT**

Chúc các bạn hoàn thành tốt bài thực hành!